HW1

1.1

(1) $7503 \bmod 81 = 51$ (2) $(-7503) \bmod 81 = 30$ $-7503 = -81 \times 92 - 51 = -81 \times 93 + 30$

(3) $81 \bmod 7503 = 81$ (4) $-81 \bmod 7503 = 7422$

1.5 BEEAKFYDJXUQYHYJIORYHTYJIQFBQDUYJIIKFUHCQD

$d_K(y) = (y - K) \bmod 26$

以 BEEAKF 为尝试点，尝试后发现当 $K = 16$ 时，B E E A F => L O O K U P

所以 $K = 16$，完整译文为：look up in the air, it's a bird, it's a plane, it's a superman.

1.6 加密函数 e_K 和解密函数 d_K 相同

则 $x + K \equiv x - K \pmod{26}$

$K \equiv -K \pmod{26}$

这等价于：$K - (-K) = k \cdot 26$ 即 $2K = 26k$ => $K = 13k$

由 $K \in [0, 25]$ 可得：$k = 0, 1$ ∴ $e_K = d_K = X$ 或 $e_K = d_K = x - 13$

1.7 由仿射密码的定义：$e_K(x) = (ax + b) \bmod m$

其中：b 有 $[0, m-1]$ 共 m 种选法，而 a 要求满足 $\gcd(a, m) = 1$

故：Sum keys $= m \times \phi(m)$

对于 $m = 30$：$30 = 2 \times 3 \times 5$ $\phi(30) = 30 - 15$（倍数）$- 5$（除倍数的 3 的倍数）$- 2$（除倍数的 3 的 5 的倍数）

$= 8$

∴ Sum keys $= 30 \times 8 = 240$

对于 $m = 100$：$100 = 2 \times 2 \times 5 \times 5$ $\phi(100) = 100 - 50$（倍数）$- 10$（除倍数的 5 的倍数）$= 40$

∴ Sum keys $= 100 \times 40 = 4000$

对于 $m = 1225$：$1225 = 5 \times 5 \times 7 \times 7$ $\phi(1225) = 1225 - (2 \times 10 \times 12 + 5)$（5 的倍数）$- 150$（除 5 倍数外 7 的倍数）$= 840$

∴ Sum keys $= 1225 \times 840 = 1029000$

1.8 都有 1、28 的逆元是自身，即 $1\times1\equiv1 \pmod{29}$，$28\times28\equiv1 \pmod{29}$

29 的倍数 +1 有：30、59、88、117、146、175

a	a^{-1}	
2	15	30
3	10	30
4	22	88
5	6	30
7	25	175
8	11	88
9	13	117
12	17	$12\times17=204=29\times7+1$
14	27	$14\times27=378=29\times13+1$
16	20	$16\times20=320=29\times11+1$
18	21	$18\times21=378=29\times13+1$
19	26	$19\times26=494=29\times17+1$
23	24	$23\times24=552=29\times19+1$.
28	28	
1	1	

1.9 (1) 已知：仿射密码 $k=(5,21)$ $\therefore e_k(x)=(5x+21) \bmod 29$.

在 Z_{29} 中，$a=5$ 对应的 $a^{-1}=6$，因为 $5\times6\equiv1 \pmod{29}$.

$\therefore d_k(y)=6(y-21) \bmod 29 = (6y+19) \bmod 29$.

$\therefore a'=6$ $b'=19$.

(2) 证明对 $\forall x\in Z_{29}$，其 $d_k(e_k(x))=x$ 成立.

$\because d_k(e_k(x))=[6\times(5x+21)+19] \bmod 29$

$=(29x+5\times29+x) \bmod 29 = [29\times(x+5)+x] \bmod 29 = x \bmod 29$

又 $\because x\in[0,28]$ $\therefore d_k(e_k(x))=x \bmod 29 = x$.

1.10

对二阶矩阵 $\begin{pmatrix} 2 & 5 \\ 9 & 5 \end{pmatrix}$，其对应的伴随矩阵是 $\begin{pmatrix} 5 & -9 \\ -5 & 2 \end{pmatrix}$

$\det(A) = 2\times5 - 45 = -35 \qquad \det(A) \equiv -35 + 26\times5 = 17 \pmod{26}$

$\therefore \det(A)^{-1} = 17^{-1} \qquad 17x \equiv 1 \pmod{26}$ 可解得：$x = 23$

$\therefore \det(A)^{-1} = 23$ 而 $A^{-1} = \det(A)^{-1} \cdot A^{*} \bmod 26 = 23 \times \begin{pmatrix} 5 & -9 \\ -5 & 2 \end{pmatrix} \bmod 26 = \begin{pmatrix} 11 & 15 \\ 1 & 20 \end{pmatrix}$

$\therefore A^{-1} = \begin{pmatrix} 11 & 15 \\ 1 & 20 \end{pmatrix}$

1.16

(1)

x	1	2	3	4	5	6	7	8
$f^{-1}(x)$	2	4	6	1	8	3	5	7

(2) 将密文按 8 个一组分组：

T G E E M N E L => $y_2\ y_4\ y_6\ y_1\ y_8\ y_3\ y_5\ y_7$ => 明文：G E N T L E M E.

N N T D R O E O => N D O N O T R E

A A H D O E T C => A D E A C H O T

S H A E I R L M => H E R S M A I L

1.18 求取密钥流的周期.

$\because (z_1, z_2, z_3, z_4) \in (Z_2)^4$

$\therefore$ 共 $2^4 = 16$ 种情况，其中考虑分为 $(0,0,0,0)$ 和其他

① $(0,0,0,0)$ 永远生成 $(0,0,0,0)$ 周期为 1

② 以 $(1,1,1,0)$ 举例 $(1,1,1,0) \to (1,1,1,0) \to (1,1,0,1) \to (1,0,1,1) \to (0,1,1,1) \to (1,1,1,0)$

可见周期为 5

其余 14 种情况经测试也符合周期为 5

1.19.

递推式: $z_{i+4} = (z_i + z_{i+3}) \pmod 2$

其对应的特征多项式为: $f(x) = x^4 + x^3 + 1$

$f(x)$是不约的，且由 $x^4 \equiv x^3 + 1 \pmod{f(x)}$ 可得: $x^5 = x \cdot x^4 \equiv x(x^3+1) = x^4 + x \equiv (x^3+1) + x = x^3 + x + 1$

$x^6 = x \cdot x^5 \equiv x(x^3+x+1) = x^4 + x^2 + x \equiv (x^3+1) + x^2 + x = x^3 + x^2 + x + 1$

$x^7 = x \cdot x^6 \equiv x(x^3+x^2+x+1) = x^4 + x^3 + x^2 + x \equiv (x^3+1) + x^2 + x^2 + x \equiv x^3 + x + 1$

$x^8 = (x^4)^2 \equiv (x^3+1)^2 = x^6 + 1 \equiv x^3 + x^2 + x$

$\therefore x^{15} = x^7 \cdot x^8 \equiv (x^3+x+1)(x^3+x^2+x)$ 化简得: $x^{15} \equiv x^5 + x^3 + x$

又 $x^5 \equiv x^3 + x + 1$ $\therefore x^{15} \equiv (x^3+x+1) + x^3 + x \equiv 1$

$\therefore x^{15} \equiv 1 \pmod{f(x)}$ $\therefore f(x)$是一个本原多项式

$\therefore$ 周期为 $2^4 - 1 = 15$

补充题:

① 极致的安全性. 在个人的项目经历中，使用过 Hash Password 来存储用户注册的密码，这个密码在数据库中是以密文的形式存储，且无法被逆向破解到用户的原密码.

② 需要能抗住算力的爆破，你存的密码算法要能抗住算力的暴力破解.

③ 强随机性？理论上攻击者可以靠让加密算法去加密他精心准备的明文字，然后观察其中的规律，你的算法要尽能避免这一点.

```python
# 标准英文字母频率表
ENGLISH_FREQS = [
    0.08167, 0.01492, 0.02782, 0.04253, 0.12702, 0.02228, 0.02015, # A-G
    0.06094, 0.06966, 0.00153, 0.00772, 0.04025, 0.02406, 0.06749, # H-N
    0.07507, 0.01929, 0.00095, 0.05987, 0.06327, 0.09056, 0.02758, # O-U
    0.00978, 0.02360, 0.00150, 0.01974, 0.00074                    # V-Z
]

# a 的有效候选值 (与26互素)
VALID_A_VALUES = [1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25]

def get_inverse(a, m=26):
    """计算 a 在模 m 下的乘法逆元"""
    return pow(a, -1, m)

def decrypt_affine(ciphertext, a, b):
    """使用指定的 a 和 b 解密仿射密文"""
    a_inv = get_inverse(a)
    plaintext = ""
    for char in ciphertext:
        if char.isalpha():
            # 将字母映射到 0-25
            y = ord(char.upper()) - ord('A')
            # 应用解密公式: P = a^-1 * (C - b) mod 26
            x = (a_inv * (y - b)) % 26
            plaintext += chr(x + ord('A'))
        else:
            plaintext += char
    return plaintext

def calculate_score(text):
    """计算文本的频率得分 (互重合指数点积)"""
    text = [c for c in text if c.isalpha()]
    if not text:
        return 0.0

    length = len(text)
    score = 0.0
    # 统计文本中每个字母的频率，并与标准英语频率求点积
    for i in range(26):
        char = chr(i + ord('A'))
        char_count = text.count(char)
        char_freq = char_count / length
        score += char_freq * ENGLISH_FREQS[i]

    return score

def crack_affine(ciphertext):
    """通过频率分析破解仿射密码"""
    best_score = 0
    best_plaintext = ""
    best_key = (0, 0)

    # 遍历所有可能的 312 种密钥组合
    for a in VALID_A_VALUES:
        for b in range(26):
            # 1. 解密
            candidate_text = decrypt_affine(ciphertext, a, b)
            # 2. 评分
            score = calculate_score(candidate_text)

            # 3. 记录最高分的解
            if score > best_score:
                best_score = score
                best_plaintext = candidate_text
                best_key = (a, b)

    return best_key, best_score, best_plaintext

# --- 测试部分 ---
if __name__ == "__main__":
    # 录入密文
    ciphertext = (
        "KQEREJEBCPPCJCRKIEACUZBKRVPKRBCIBQCARBJCVFCUP"
        "KRIOFKPACUZQEPBKRXPEIIEABDKPBCPFCDCCAFIEABDKP"
        "BCPFEQPKAZBKRHAIBKAPCCIBURCCDKDCCJCIDFUIXPAFF"
        "ERBICZDFKABICBBENEFCUPJCVKABPCYDCCDPKBCOCPERK"
        "IVKSCPICBRKIJPKABI"
    )

    best_key, score, plaintext = crack_affine(ciphertext)

    print(f"找到最优密钥: a = {best_key[0]}, b = {best_key[1]}")
    print("\n解密后的明文:")
    print(plaintext)
```

```
(base) lixinyang@LxyMac 实习 % python -u "/Users/lixinyang/Desktop/实习/1.py"
找到最优密钥: a = 19, b = 4

解密后的明文:
OCANADATERREDENOSAIEUXTONFRONTESTCEINTDEFLEURONSGLORIEUXCARTONBRASSAITPORTERLEPEEILSAITPORTERLACROIXTONHISTOIREESTUNEEPOPEEDESPLUSBRILLANTSEXPLOITSETTAVALEURDEFOITREMPEEPROTEGERANOSFOYERSETNOSDROITS
(base) lixinyang@LxyMac 实习 %
```

经查询翻译出的明文是 Canada 的国歌.